

Trabajo Práctico Obligatorio (TPO)

Propuesta elegida: Propuesta 3

Sistema de Gestión Hotelera y Servicios para Huéspedes

Materia: Proceso de Desarrollo de Software

Docente: María Ángela León

Grupo: TeamVita

Integrantes:

Joaquín Manuel De Luca

Lautaro Vita

Claudino Diaz

Fecha de entrega: 10 de Junio de 2026

Repositorio Git: <https://github.com/joaacoo/TeamVita-sistema-hotelero.git>

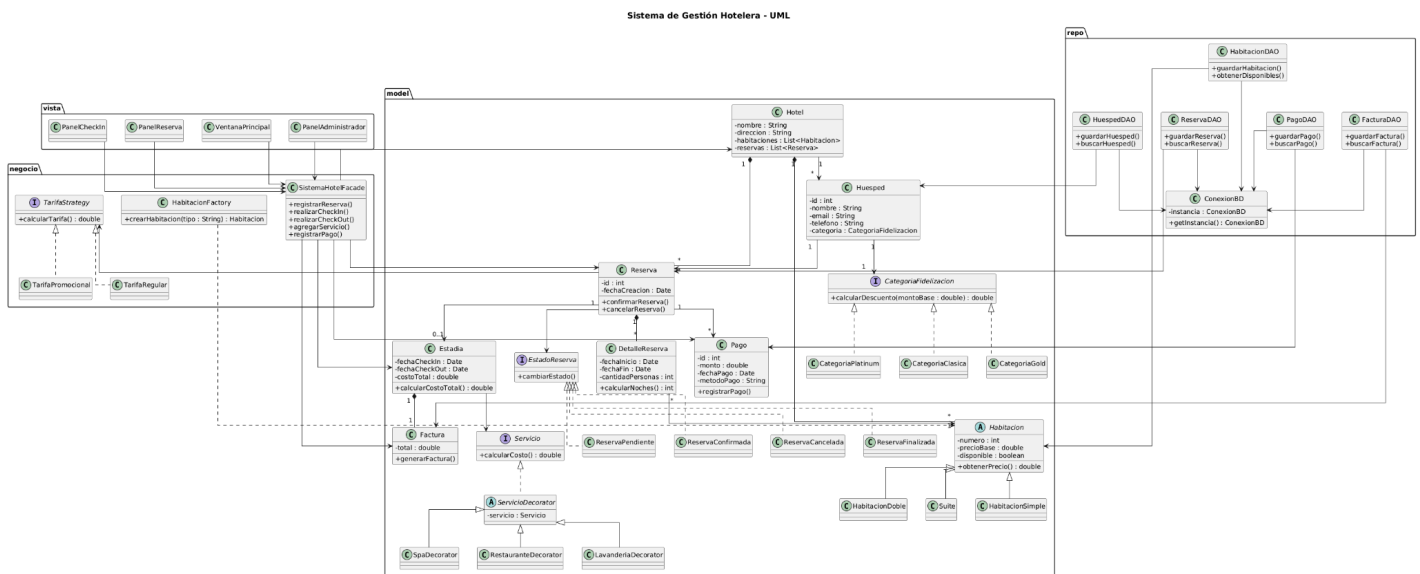
UADE

Fase 2: Análisis del problema y diseño inicial

A partir del modelo inicial que planteamos en la Fase 1, detectamos que la clase Reserva estaba asumiendo demasiadas responsabilidades y violando el principio SRP (se estaba convirtiendo en una Clase Dios). Por eso, para esta Fase 2, decidimos refactorizarla y extraer la lógica de ocupación a la nueva clase DetalleReserva. Además, para mejorar el modelo del Huésped, incorporamos el patrón Strategy de fidelización.

1. Diagrama de clases UML

Se adjunta el diagrama de clases correspondiente a la arquitectura del sistema, dividido en las capas funcionales: vista, negocio, modelo y repo.



1.1 Descripción de la arquitectura del sistema El sistema está estructurado bajo una Arquitectura en Capas, lo que nos permite separar claramente las responsabilidades y proteger el modelo de dominio. El flujo de información siempre va de arriba hacia abajo:

- **Capa de Presentación (Vista):** Contiene las interfaces gráficas con las que interactúa el usuario (recepcionista, admin).
- **Capa Lógica (Negocio):** Actúa como intermediaria, validando reglas a través de controladores y fachadas.
- **Capa de Dominio (Model):** Contiene el corazón del sistema, las entidades puras y la lógica de negocio estricta.
- **Capa de Persistencia (Repo):** Se encarga exclusivamente de la interacción con la base de datos a través de los DAOs.

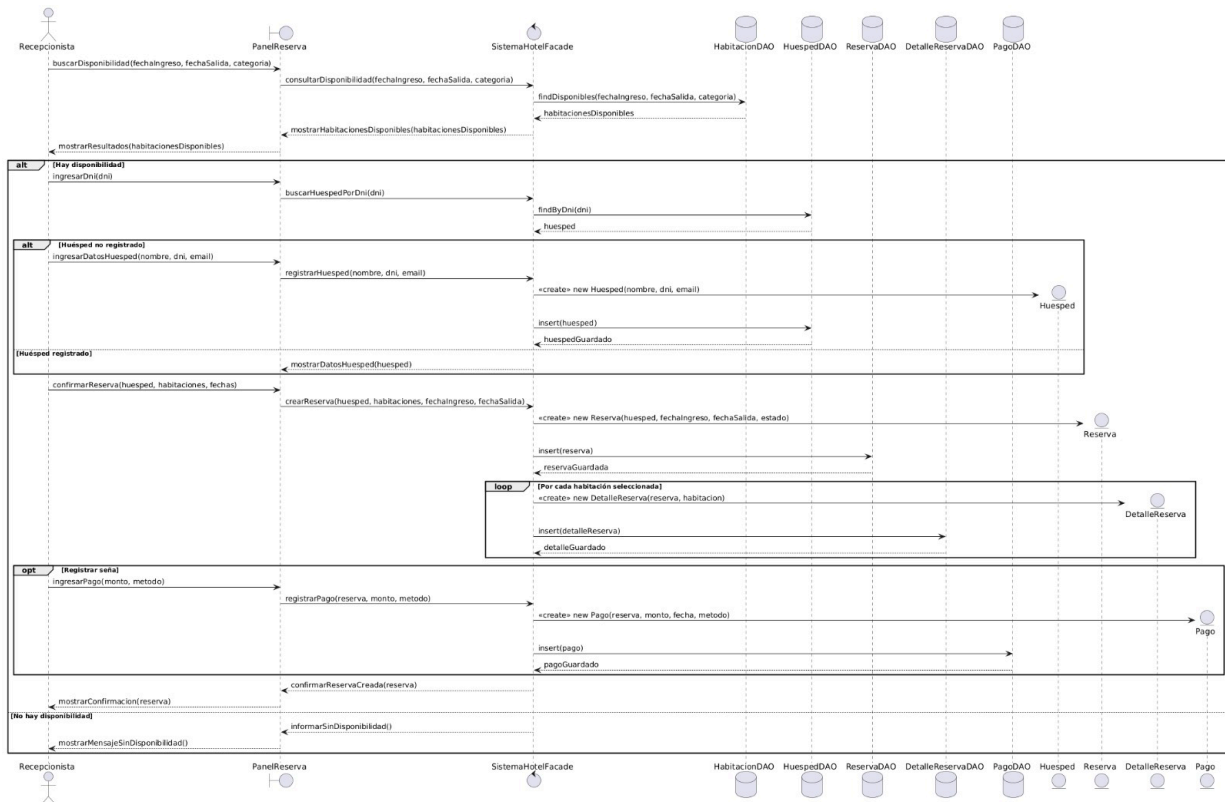
1.2 Explicación de las clases principales:

- **Hotel:** Actúa como la clase contenedora y gestora general de los recursos principales del sistema.
- **Huesped:** Almacena la información vital del cliente y gestiona su categoría de fidelización para interactuar con las reglas de negocio.
- **Estadia:** Representa el ciclo de vida real del alojamiento, desde el check-in hasta la facturación, vinculando los servicios consumidos y la habitación ocupada.
- **Reserva y DetalleReserva:** Manejan la intención previa de alojamiento, separando la parte administrativa (fechas, titular) de la especificación física (tipo de habitación).

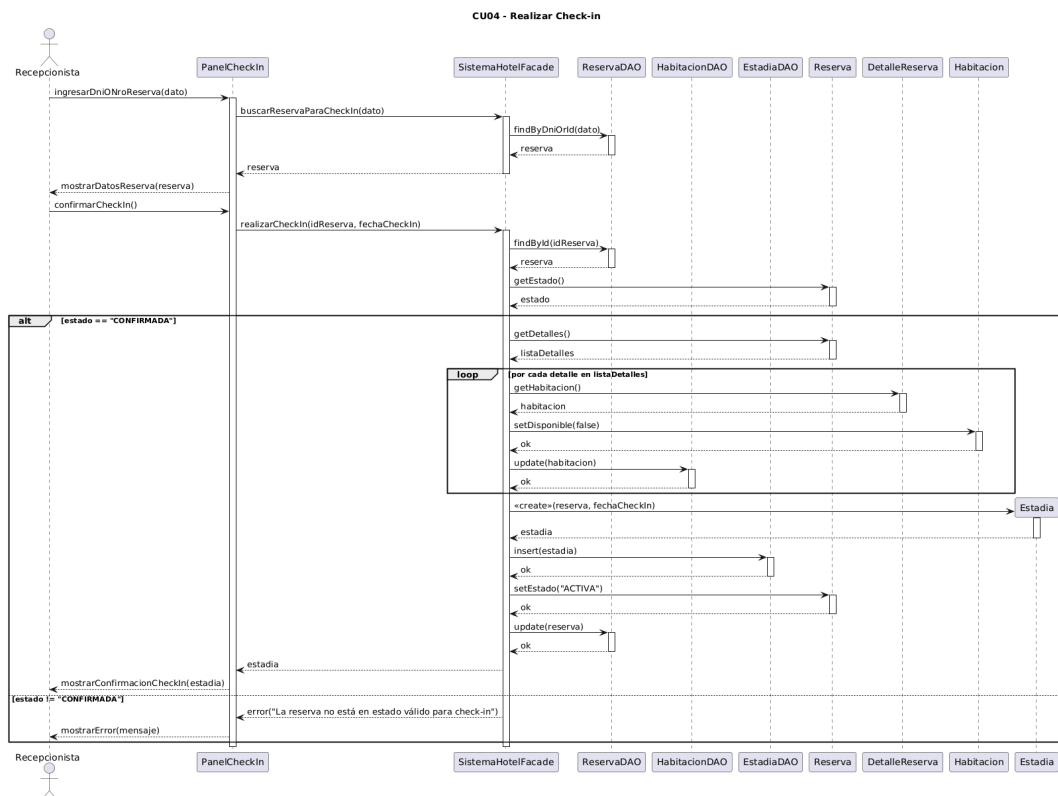
2. Diagrama de secuencia de casos de uso relevantes

En esta sección presentamos en detalle el funcionamiento de los dos casos de uso más representativos del sistema:

1. **CU02 - Gestionar Reserva (Creación):** El recepcionista interactúa con el PanelReserva. Éste delega a la clase SistemaHotelFacade la verificación de disponibilidad a través de HabitacionDAO. Al confirmar disponibilidad y registrar/validar al Huesped (vía HuespedDAO), la Fachada instancia la Reserva y la persiste usando ReservaDAO. El estado inicial de la reserva se gestiona internamente al momento de la creación.



2. **CU04 - Realizar Check-in:** Desde el PanelCheckIn, se solicita el ingreso al hotel. SistemaHotelFacade recupera la Reserva con ReservaDAO. Se verifica su estado mediante el patrón State (EstadoReserva). Si el estado permite el ingreso (ej. ReservaConfirmada), se iteran las habitaciones asociadas, marcándolas como no disponibles a través del HabitacionDAO. Finalmente, se instancia la Estadia, se actualiza el estado de la reserva, y se persiste el cambio.



3. Identificación de patrones de diseño a aplicar y 4. Justificación

Para resolver la problemática planteada y garantizar una arquitectura flexible, se seleccionaron los siguientes patrones de diseño:

- **Patrones creacionales:**

1. **Factory Method (HabitacionFactory):** Centraliza y encapsula la lógica de creación de las distintas variantes de habitaciones (Suite, HabitacionSimple, HabitacionDoble). Esto evita que la Fachada esté fuertemente acoplada a las clases concretas. De instanciar directamente con `new Suite()` en el código, la capa de vista o el controlador estarían fuertemente acoplados. El uso del Factory nos permite que el día de mañana el hotel agregue una nueva categoría (ej. 'HabitacionPresidencial') sin tener que tocar la lógica core de reservas.
2. **Singleton (ConexionBD):** Garantiza que exista una única instancia de conexión a la base de datos MySQL, optimizando el uso de recursos y centralizando el punto de acceso para todos los DAOs en la capa repo.

- **Patrones estructurales:**

1. **Facade (SistemaHotelFacade):** Proporciona un punto de entrada unificado y simplificado desde el paquete vista hacia la lógica de negocio y model. Oculta la complejidad de interactuar con múltiples componentes del dominio y los repositorios de datos.
2. **Decorator (ServicioDecorator):** Permite añadir responsabilidades o servicios adicionales (SpaDecorator, RestauranteDecorator, LavanderiaDecorator) a la estadía de forma dinámica. Evita la proliferación de subclasses para cada combinación posible de servicios contratados.

- **Patrones de comportamiento:**

1. **Strategy (TarifaStrategy):** Encapsula los algoritmos de cálculo de precios (TarifaPromocional, TarifaRegular, TarifaCorporativa, TarifaTemporadaAlta), permitiendo cambiar la política comercial en tiempo de ejecución sin alterar el código de la reserva o estadía. Sin este patrón, la clase Estadía tendría múltiples bloques ifs para calcular costos. Cada vez que marketing lanzara una nueva promoción, tendríamos que modificar el código central del sistema, violando el principio Open/Closed.
2. **State (EstadoReserva):** Administra las transiciones lógicas de la reserva (ReservaPendiente, ReservaConfirmada, ReservaCancelada, ReservaFinalizada). Delega el comportamiento dependiente del estado a clases concretas, eliminando bloques condicionales extensos y propensos a

errores.

3. **Strategy (CategoriaFidelizacion):** Se aplica este patrón en la clase Huesped para encapsular el algoritmo de cálculo de descuentos según su nivel de membresía (Clásica, Gold, Platinum). Esto le otorga al huésped un rol activo en las reglas de negocio y permite asignar beneficios dinámicos en la facturación sin llenar el código de sentencias condicionales.

5. Identificación de principios SOLID aplicados:

1. **Single Responsibility Principle (SRP):** Cada clase del sistema tiene una única responsabilidad bien definida. Las clases del paquete model (como Huesped o Reserva) se encargan exclusivamente de representar el dominio, mientras que las clases del paquete repo (como FacturaDAO) son las únicas responsables de interactuar con la base de datos. El ejemplo más claro de la aplicación de este principio fue la refactorización de la clase Reserva, la cual fue dividida en Reserva (responsable del ciclo de vida y datos administrativos) y DetalleReserva (responsable de la especificación de la habitación, capacidad y fechas exactas de ocupación). Esto evitó la creación de un anti-patrón de 'Clase Dios'.
2. **Open/Closed Principle (OCP):** La arquitectura está diseñada para ser abierta a la extensión pero cerrada a la modificación. Gracias al patrón Decorator, se pueden incorporar nuevos servicios adicionales al hotel sin necesidad de modificar el código existente en las clases de facturación o estadía.
3. **Liskov Substitution Principle (LSP):** Las clases derivadas pueden sustituir a sus clases base sin alterar el correcto funcionamiento del sistema. Esto se evidencia claramente en la jerarquía de habitaciones: en cualquier proceso que requiera operar con un objeto del tipo base Habitacion (por ejemplo, al calcular el precio base), se le puede suministrar una instancia de Suite, HabitacionSimple o HabitacionDoble, y el sistema responderá correctamente a través del polimorfismo.
4. **Dependency Inversion Principle (DIP):** El modelo de dominio cumple fuertemente con este principio al depender de abstracciones (interfaces) y no de implementaciones concretas. Por ejemplo, en lugar de que la clase Reserva o Estadía dependa directamente de una clase concreta como TarifaTemporadaAlta, depende de la interfaz pura TarifaStrategy. De esta forma, el módulo de alto nivel (el cálculo de la estadía) invierte su dependencia hacia una abstracción, permitiendo agregar nuevas lógicas de precios sin alterar el código que las consume. Lo mismo ocurre con la interfaz EstadoReserva.
5. **Interface Segregation Principle (ISP):** Se diseñaron interfaces pequeñas y de propósito único, en lugar de crear "Interfaces Dios". Un claro ejemplo es la interfaz TarifaStrategy, la cual expone únicamente el método calcularTarifa(). De este modo, cualquier clase cliente o nueva estrategia que se implemente no se ve forzada a depender o implementar métodos que no necesita.

6. Identificación de patrones GRASP aplicados:

1. **Controlador (Controller):** La clase SistemaHotelFacade actúa como el controlador principal que recibe, coordina y delega las operaciones del sistema solicitadas por la interfaz gráfica.
2. **Experto en Información (Information Expert):** La clase Estadia posee la responsabilidad de calcularCostoTotal() ya que es la experta en información: es la única que tiene acceso a los días alojados, la reserva asociada y los servicios adicionales consumidos por el huésped.
3. **Alta Cohesión y Bajo Acoplamiento:** La separación estricta en capas (vista, negocio, model y repo) garantiza que cada paquete trabaje con elementos estrechamente relacionados (alta cohesión) minimizando las dependencias directas con el funcionamiento interno de las demás capas (bajo acoplamiento).
4. **Polimorfismo (Polymorphism):** Se asignan responsabilidades utilizando operaciones polimórficas para manejar el comportamiento que varía según el tipo. Esto es clave en el sistema y se evidencia al usar las interfaces TarifaStrategy, EstadoReserva y CategoriaFidelizacion. En lugar de tener una clase central con múltiples if/switch para saber cómo calcular un descuento o validar un cambio de estado, el sistema delega la ejecución al objeto polimórfico correspondiente.
5. **Fabricación Pura (Pure Fabrication):** Para no violar la Alta Cohesión y el Bajo Acoplamiento, se crearon clases que no representan conceptos del dominio real del problema (en un hotel físico no existe un "DAO" o una "Conexión a BD"). Las clases SistemaHotelFacade, ConexionBD, HabitacionFactory y todos los DAOs son Fabricaciones Puras introducidas artificialmente en el diseño de software para organizar el código y aislar responsabilidades.
6. **Creador (Creator):** Se aplica este patrón al asignar la responsabilidad de instanciar los objetos complejos. Por ejemplo, el SistemaHotelFacade actúa como Creador de la Reserva y la Estadia, ya que es la clase que recolecta y posee los datos de inicialización enviados desde la interfaz gráfica (Panel de Recepción) y tiene la responsabilidad de registrar esa instancia en la base de datos a través del DAO.
7. **Indirección (Indirection):** Para evitar el acoplamiento directo entre la capa de Presentación (Vista) y la capa de Dominio o Persistencia, se introdujo el SistemaHotelFacade. Esta fachada actúa como un componente intermediario (indirección). Gracias a esto, un PanelReserva no interactúa directamente con HabitacionDAO o con la entidad Reserva, sino que le envía el mensaje a la Fachada, logrando un bajo acoplamiento.
8. **Variaciones Protegidas (Protected Variations):** El sistema está diseñado para que las variaciones y la inestabilidad en ciertas lógicas no afecten al resto del sistema. Al encapsular el cálculo de tarifas detrás de la interfaz TarifaStrategy, el núcleo de la Estadia queda "protegido". Si mañana el hotel decide crear una nueva política de precios (una variación), el sistema no se rompe ni requiere modificar el código.

existente; simplemente se crea una nueva clase que implemente la interfaz.

7. Estructura inicial de paquetes del proyecto Java

El proyecto se encuentra organizado en el directorio raíz `src/main/java/com/teamvita/hotel/` y está dividido en cuatro paquetes principales:

1. Paquete model (Capa de Dominio):

- Entidades principales: `Hotel.java`, `Huesped.java`, `Reserva.java`, `DetalleReserva.java`, `Estadia.java`, `Pago.java`, `Factura.java`.
- Clases base de herencia: `Habitacion.java` (junto con sus subclases `HabitacionSimple`, `HabitacionDoble`, `Suite`).
- Patrones aplicados: `CategoriaFidelizacion.java` (Strategy), `EstadoReserva.java` (State) y `Servicio.java` (Decorator).

2. Paquete negocio (Capa Lógica):

- Controlador principal: `SistemaHotelFacade.java`.
- Patrones aplicados: `HabitacionFactory.java` (Factory) y `TarifaStrategy.java` (Strategy para cálculos).

3. Paquete repo (Capa de Persistencia):

- Base de datos: `ConexionBD.java` (Patrón Singleton).
- Clases de acceso a datos: `ReservaDAO.java`, `HuespedDAO.java`, `HabitacionDAO.java`, `PagoDAO.java`, `FacturaDAO.java`.

4. Paquete vista (Capa de Presentación):

- Interfaces gráficas: `VentanaPrincipal.java`, `PanelReserva.java`, `PanelCheckIn.java`, `PanelAdministrador.java`.

8. Primera versión del código (Clases principales e interfaces)

El código fuente correspondiente a esta fase, estructurado en los paquetes mencionados y con la implementación inicial de las clases principales, interfaces y relaciones, se encuentra versionado en nuestro repositorio de control de versiones. Se puede acceder al repositorio en el siguiente enlace: <https://github.com/joaacoo/TeamVita-sistema-hotelero.git>.

Para evidenciar la integración de los componentes desarrollados en esta fase, a continuación se presentan capturas de ejecución de la interfaz gráfica, demostrando cómo el controlador `SistemaHotelFacade` conecta exitosamente la vista (como el `PanelReserva` y `PanelCheckIn`) con los DAOs correspondientes.

TeamVita - Sistema de Gestión Hotelera - Usuario: admin

Administración

Dashboard de Administración

Ocupación Actual

0%

Habitaciones Libres

3

Ingresos Totales

\$ 0,00

Habitaciones y Tarifas

Servicios Extra y Config.

Promociones

Gestión de Habitaciones y Tarifas

Número	Tipo	Precio Base	Capacidad	Disponible
101	Simple	\$ 50,00	1	Sí
102	Doble	\$ 80,00	2	Sí
201	Suite	\$ 150,00	6	Sí

Nueva Habitación

Editar Habitación

TeamVita - Sistema de Gestión Hotelera - Usuario: recepcion

Nueva Reserva

Gestionar Reservas

Registrar Nueva Reserva

DNI Huésped (Máx 8):

Nombre Completo:

Correo Electrónico:

Teléfono:

Categoría Fidelización:

Clásica

Check-In:

...

Check-Out:

...

Habitación Disponible:

101 - Simple (\$50.0 | cap: 1)

Cantidad de Personas:

1

Promoción:

Sin Promoción

Confirmar reserva

8

9. Informe breve de avance individual por integrante

- **Joaquín De Luca:** Configuración inicial del repositorio y del entorno Java. Encargado de la programación completa de la capa de dominio (model), definiendo las entidades puras, sus relaciones y la justificación teórica de los principios SOLID implementados (SRP, OCP, LSP).
- **Lautaro Vita:** Responsable de la capa de negocio. Implementación del controlador principal aplicando el patrón Facade (SistemaHotelFacade) y estructuración de los patrones creacionales y de comportamiento (Factory Method y Strategy).
- **Claudino Diaz:** A cargo del diseño y programación de la capa de acceso a datos (repo), definiendo las interfaces y simulando la persistencia de los DAOs. Responsable de implementar el patrón Singleton para la conexión a la base de datos y documentar los patrones GRASP.